

Автономная некоммерческая организация профессиональная образовательная
организация

«Колледж экономики, права и информационных технологий»

(АНО ПОО «КЭПиИТ»)

Курсовой проект

«Разработка веб-приложения распределенного мониторинга внутренней инфраструктуры «Upward»»

(тема курсового проекта)

Специальность

09.02.07

**Информационные системы и
программирование**

(код)

(наименование)

Квалификация

Разработчик веб и мультимедийных приложений

(название квалификации)

Обучающийся

Шахсинов М. В.

—

(подпись)

(Фамилия и Инициалы)

Руководитель курсового
проекта

Двораковский Б. М.

—

(подпись)

(ученая степень, звание, Фамилия и Инициалы)

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1. ПРЕДПРОЕКТНОЕ ИССЛЕДОВАНИЕ.....	6
1.1. Описание предметной области.....	6
1.2. Описание технологии обработки задач.....	9
1.3. Информационно-логическая модель и моделирование процессов в нотации IDEF0	12
1.4. Характеристика инструментальных средств разработки.....	24
ГЛАВА 2. РАБОЧИЙ ПРОЕКТ.....	28
2.1. Анализ требований и разработка спецификаций.....	28
2.2. Разработка веб-приложения распределенного мониторинга.....	31
2.3. Отладка и тестирование веб-приложения.....	37
2.4. Руководство программиста и эксплуатация.....	39
ЗАКЛЮЧЕНИЕ.....	43
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	46
ПРИЛОЖЕНИЕ А. Модуль фонового распределенного опроса целевых ресурсов.....	48
ПРИЛОЖЕНИЕ Б. Многоуровневый валидатор сетевой безопасности и защиты от SSRF.....	49
ПРИЛОЖЕНИЕ В. Схема базы данных (DDL миграции TimescaleDB).....	50

ВВЕДЕНИЕ

В условиях перехода современных предприятий на микросервисную архитектуру и распределенные веб-системы обеспечение непрерывной доступности (uptime) сервисов становится ключевым условием стабильности корпоративной инфраструктуры. Простои веб-узлов, сбои прикладных программных интерфейсов (API) и неконтролируемые сетевые задержки ведут к деградации бизнес-процессов, снижению лояльности клиентов и прямым финансовым потерям организаций.

До настоящего времени задачи контроля жизнеспособности сервисов преимущественно решались с использованием зарубежных коммерческих SaaS-платформ (таких как UptimeRobot, BetterStack, Pingdom) либо тяжеловесных систем корпоративного мониторинга вроде Zabbix, Nagios и Prometheus. Однако публичные облачные SaaS-сервисы архитектурно не способны контролировать закрытые внутренние контуры (виртуальные частные сети VPN, Intranet, демилитаризованные зоны DMZ) без создания критических брешей в политиках информационной безопасности. Кроме того, передача внутренних сетевых адресов и метаданных сторонним облачным провайдерам несет существенные риски утечки данных, регуляторных нарушений и блокировок доступа. Традиционные же платформы мониторинга ориентированы на сбор метрик серверного «железа», чрезмерно сложны в конфигурировании, лишены встроенных механизмов строгой изоляции профилей пользователей и затрудняют сбор кастомной телеметрии непосредственно от инспектируемых веб-приложений.

Актуальность темы обусловлена необходимостью создания независимого, высокопроизводительного open-source решения «Upward» для uptime-мониторинга внутренней инфраструктуры предприятий, предназначенного для автономного (self-hosted) развертывания с разграничением профилей пользователей, поддержкой кастомных метрик и надежной защитой от сетевых

атак.

Объектом исследования выступают процессы непрерывного контроля доступности, замера временных задержек и сбора телеметрии внутренних веб-сервисов организации.

Предметом исследования являются методы, архитектурные паттерны и программные средства проектирования распределенной self-hosted системы мониторинга высокой доступности.

Целью проекта является проектирование и разработка распределенного веб-приложения мониторинга доступности внутренней инфраструктуры «Upward», обеспечивающего субмиллисекундную скорость опроса, строгую изоляцию профилей пользователей, прием кастомных данных телеметрии и устойчивость к высоким сетевым нагрузкам.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать предметную область, существующие программные аналоги и специфику мониторинга закрытых инфраструктурных контуров;
- спроектировать микросервисную архитектуру системы, включающую L7-балансировку на базе Nginx и асинхронную шину взаимодействия;
- разработать входной шлюз Backend-for-Frontend (BFF) на платформе NestJS/TypeScript для агрегации клиентских запросов;
- разработать сервис аутентификации и профилей (Identify) на языке Rust с реализацией асимметричной криптографической схемы подписи токенов (Ed25519/RSA), при которой закрытый ключ хранится в изолированной памяти сервиса, а открытый ключ предоставляется остальным микросервисам для независимой валидации;
- разработать высокопроизводительный, масштабируемый сервис периодического опроса Uptime на языке Rust с использованием асинхронного рантайма Tokio и фреймворка Axum;
- реализовать механизмы многоуровневой защиты от атак класса Server-

Side Request Forgery (SSRF) и подтверждения владения ресурсами через протокол HTTP-01 Challenge;

- внедрить специализированную реляционную систему управления базами данных PostgreSQL с расширением TimescaleDB для хранения временных рядов замеров с автоматической политикой ротации данных (retention policy);

- выполнить комплексную отладку, нагрузочное и функциональное тестирование программных компонентов.

Методы и источниковая база исследования базируются на системном анализе, теории распределенных вычислений, протоколах HTTP/HTTPS, TCP/IP, асинхронном неблокирующем вводе-выводе (epoll), официальной технической документации языков Rust и TypeScript, рантайма Tokio, фреймворков Axum и NestJS, СУБД TimescaleDB, а также публикациях ведущих специалистов в области высоконагруженных сетевых сервисов.

Функциональное назначение «Upward» заключается в непрерывном автоматизированном контроле доступности веб-узлов, сборе временных рядов сетевой задержки, изоляции данных профилей пользователей, верификации прав на отслеживаемые ресурсы и предоставлении интерактивного интерфейса управления через OpenAPI/Swagger UI.

Практическая значимость работы заключается в создании готового к промышленному внедрению программного комплекса, свободного от лицензионных ограничений, развертываемого в закрытом контуре компании и обеспечивающего суверенитет и конфиденциальность корпоративных данных.

Инструментальные средства проекта включают: Rust (Tokio, Axum, SQLx), TypeScript/NestJS, PostgreSQL/TimescaleDB, Redis, Nginx Reverse Proxy, Nix, devenv и Moonrepo. Пояснительная записка состоит из введения, двух глав, заключения, списка использованных источников и приложений.

ГЛАВА 1. ПРЕДПРОЕКТНОЕ ИССЛЕДОВАНИЕ

1.1. Описание предметной области

Современные информационные системы крупных технологических предприятий и финансовых организаций (FinTech) строятся на базе распределенной микросервисной архитектуры [1]. В таких экосистемах прикладная бизнес-логика декомпозируется на десятки и сотни автономных веб-сервисов: платежные шлюзы, подсистемы процессинга транзакций, сервисы скоринга, фронтальные API-шлюзы для мобильных клиентов и внутренние административные порталы. Обеспечение непрерывной работоспособности (High Availability) каждого компонента выступает базовым критерием стабильности бизнеса, регламентируемым соглашениями об уровне обслуживания (Service Level Agreement, SLA) с целевыми показателями доступности не ниже 99.9%–99.99%.

Предметная область данного исследования охватывает процессы автоматизированного мониторинга состояния сетевой инфраструктуры (uptime-мониторинг), периодического зондирования веб-узлов по протоколам HTTP/HTTPS, сбора и аналитической обработки телеметрических показателей сетевой задержки (Latency/Round Trip Time, RTT), а также оперативного оповещения ответственного персонала о деградации или недоступности критически важных сервисов.

В качестве базовой организации для внедрения разрабатываемой системы рассматривается технологическая финтех-компания, обладающая разветвленной гетерогенной ИТ-инфраструктурой. Организационная структура компании включает инженерный департамент платформенных сервисов, отдел надежности инфраструктуры (Site Reliability Engineering, SRE), команды продуктовой бэкенд-разработки и службу информационной безопасности (Security Operations Center, SOC). Корпоративная сеть разделена на несколько

изолированных сетевых зон:

1. Демилитаризованная зона (DMZ) — внешний контур, содержащий обратные прокси-серверы (Reverse Proxy), веб-серверы и внешние точки входа для клиентских мобильных и веб-приложений.

2. Внутренний корпоративный контур (Intranet) — закрытый периметр, в котором размещены микросервисы бизнес-логики, базы данных, очереди сообщений и брокеры очередей. Доступ в данный контур извне напрямую заблокирован межсетевыми экранами (Firewall).

3. Виртуальные частные сети (VPN) — защищенные криптографические туннели, используемые распределенными командами инженеров для администрирования инфраструктуры и отладки сервисов.

До внедрения программного комплекса «Upward» задачи контроля доступности в организации решались комбинацией двух неэффективных подходов:

– Использование сторонних зарубежных SaaS-платформ (UptimeRobot, BetterStack) для внешних веб-узлов. Данный подход доказал свою принципиальную неприменимость для закрытого контура, поскольку мониторинг внутренних микросервисов требовал бы открытия внешних портов в корпоративном брандмауэре, что категорически запрещено требованиями регуляторов и стандартами безопасности банковской тайны и персональных данных (ФЗ № 152 «О персональных данных», ГОСТ Р 56939-2016) [2]. Кроме того, возникли существенные геополитические риски внезапного одностороннего прекращения обслуживания со стороны зарубежных провайдеров.

– Применение самописных скриптов на базе утилит cURL и ping, запускаемых по расписанию cron на серверах дежурных администраторов. Недостатками скриптового подхода являлись отсутствие централизованной базы данных временных рядов, невозможность горизонтального масштабирования на тысячи узлов, отсутствие разграничения прав доступа

между командами, высокий риск исчерпания файловых дескрипторов операционной системы при одновременном запуске сотен блокирующих процессов, а также отсутствие визуального интерфейса для анализа динамики задержек.

Внедрение программного продукта «Upward» кардинально трансформирует бизнес-процессы мониторинга организации:

- обеспечивается стопроцентный суверенитет данных за счет локального (self-hosted) развертывания всех компонентов в доверенном контуре организации без трансграничной передачи сетевой топологии и метаданных;

- устраняются риски утечек информации и атак типа SSRF за счет встроенного строгой многоуровневой фильтрации сетевых адресов;

- внедряется модель многопользовательской изоляции (Multi-Tenancy): каждая продуктовая команда регистрирует и отслеживает только закрепленные за ней ресурсы, сохраняя при этом общую централизованную инфраструктуру опроса;

- за счет применения реактивного асинхронного неблокирующего ввода-вывода (Tokio/epoll) достигается способность выполнения десятков тысяч проверок в минуту при минимальном потреблении вычислительных ресурсов сервера (менее 50 МБ оперативной памяти);

- автоматизируется сбор исторических временных рядов с глубиной хранения 30 дней и автоматической очисткой устаревших партиций, что высвобождает до 30% рабочего времени дежурных SRE-инженеров, ранее затрачиваемого на ручную локализацию сбоев.

Основными категориями пользователей разрабатываемого веб-приложения выступают:

- SRE- и DevOps-инженеры (возраст 22–45 лет, высокая техническая квалификация): настройка глобальных политик опроса, мониторинг выполнения SLA, конфигурирование пула проверок и анализ стабильности кластеров;

- Разработчики микросервисов (возраст 20–40 лет, средняя и высокая техническая квалификация): добавление новых сервисов в мониторинг в процессе CI/CD, верификация владения эндпоинтами, анализ метрик сетевого времени ответа (`duration_ms`) после релизов;

- Системные администраторы и специалисты безопасности: аудит прав доступа, контроль списков запрещенных к зондированию сетевых зон (черные списки IP).

Уровень готовности пользователей к внедрению продукта оценивается как высокий, поскольку система предоставляет стандартизированный программный интерфейс REST API и документацию OpenAPI (Swagger UI), легко интегрируемые в существующие корпоративные пайплайны CI/CD и дашборды мониторинга.

1.2. Описание технологии обработки задач

Технология обработки задач в распределенной системе мониторинга «Upward» базируется на циклическом конвейере: от регистрации и валидации целевого ресурса до параллельного неблокирующего зондирования и агрегированной фиксации результатов в темпоральной базе данных.

Описание входной информации включает в себя структурированные сообщения двух типов:

1. Входная информация администрирования и регистрации ресурсов:

- ``user_id``: уникальный строковый идентификатор владельца ресурса (передается из JWT-токена аутентификации, например: `'usr_01J8ABCDEF1234567890'`);

- ``site``: целевой унифицированный указатель ресурса (URL), подлежащий мониторингу. К строке URL предъявляются строгие требования валидации схемы: разрешены исключительно протоколы `'http://'` и `'https://'`. Использование схем `'file://'`, `'ftp://'`, `'gopher://'` жестко блокируется на уровне контроллера для предотвращения атак обхода периметра;

- ``challenge_token``: 128-битный псевдослучайный токен формата UUIDv4, генерируемый системой и передаваемый пользователю для подтверждения владения сайтом;

- поисковые и фильтрационные параметры HTTP-запросов: диапазон дат, идентификаторы сайтов, лимиты выборок при запросе истории замеров.

2. Входная телеметрическая информация сетевого зондирования:

- сетевые ответы опрашиваемых веб-узлов: числовой HTTP-код состояния (200, 404, 500 и др.), временной интервал отклика (`duration_ms`), заголовки ответов (Server, Content-Type) и структурированные сообщения об ошибках (DNS resolving failure, Connection Timeout, Connection Refused).

Описание выходной информации включает:

- спецификацию созданного ресурса: JSON-объект, содержащий уникальный сгенерированный UUID сайта (``id``), статус первичной регистрации (``pending_verification``), токен подтверждения (``challenge_token``) и контрольный путь (``challenge_path` = "/.well-known/upward"`);

- структурированный реестр отслеживаемых сайтов конкретного пользователя: массив сущностей с атрибутами ``id``, ``url``, ``user_id``, ``created_at``, временной меткой последнего раунда проверки (``last_check``), текущим статусом в очереди воркеров (``status``: `'idle'` | `'processing'`), флагом активности (``active``: `true` | `false`) и метаданными последнего замера в поле ``extra``;

- временные ряды метрик доступности (Site Pings): выборка исторических замеров за последние 30 суток, содержащая таймштамп ``time``, идентификатор сайта ``site_id``, задержку ``duration_ms`` с точностью до десятых долей миллисекунды и JSON-объект ``extra``;

- диагностические сообщения подсистемы самодиагностики (``GET /health``): статус готовности сервиса (``status``: `'ok'`) и текстовое подтверждение работоспособности пула соединений с БД и Redis;

- интерактивную спецификацию интерфейса OpenAPI v3.1, автоматически транслируемую по адресу ``/docs``.

Требования к программному обеспечению делятся на функциональные и эксплуатационные (нефункциональные).

Функциональные требования к программному комплексу:

- Регистрация и управление жизненным циклом отслеживаемых веб-ресурсов;
- Подтверждение легитимности владения ресурсом (HTTP-01 Challenge) через временное хранение токенов в Redis с временем жизни (TTL) 24 часа;
- Многоуровневая валидация входящих URL и предотвращение атак Server-Side Request Forgery (SSRF) с обязательным DNS-резолвингом, проверкой принадлежности к приватным подсетям RFC 1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), петлевым интерфейсам (127.0.0.0/8, ::1) и сверкой с динамическим черным списком ``forbidden_ip``;
- Фоновый конкурентный опрос сайтов с настраиваемым размером пакета (батча) и контролируемым лимитом одновременных сетевых соединений (concurrency limit);
- Исключение состояний гонки (Race Conditions) между параллельными репликами сервиса Uptime за счет атомарной блокировки строк ``FOR UPDATE SKIP LOCKED``;
- Автоматическое восстановление зависших проверок (Dead Worker Recovery) при аварийном завершении процессов-воркеров (таймаут зависания 3 минуты);
- Высокопроизводительная пакетная вставка результатов (Batch Insert) в один сетевой запрос к базе данных;
- Автоматическое партиционирование и удаление устаревших данных мониторинга старше 30 дней посредством политик TimescaleDB (Retention Policy);
- Разграничение доступа к метрикам и сайтам на основе идентификатора владельца ``user_id``.

Эксплуатационные (нефункциональные) требования:

- Производительность: время отклика REST API сервиса Uptime не должно превышать 15 миллисекунд (без учета задержки опрашиваемого узла);
- Энергоэффективность и потребление ОЗУ: базовое потребление оперативной памяти микросервисом Uptime не должно превышать 50 МБайт при поддержании 10 000 активных целевых ресурсов;
- Горизонтальная масштабируемость: архитектура сервиса должна поддерживать запуск произвольного количества параллельных реплик за L7-балансировщиком Nginx без дублирования проверок;
- Безопасность криптографических операций: валидация токенов доступа JWT должна опираться на асимметричную цифровую подпись (алгоритм Ed25519 или RSA-256), где закрытый ключ подписи надежно изолирован в памяти сервиса Identify, а открытый ключ доступен остальным микросервисам (BFF, Uptime) для мгновенной локальной проверки без дополнительных сетевых запросов;
- Воспроизводимость среды исполнения: сборка и развертывание компонентов должны быть детерминированы через декларативные профили Nix, devenv и оркестратор задач Moonrepo [3].

1.3. Информационно-логическая модель и моделирование процессов в нотации IDEF0

Информационно-логическая модель распределенного комплекса «Upward» отражает функциональную декомпозицию системы на микросервисы, потоки данных между ними, структуру долговременного хранения телеметрии и формализованное описание процессов в нотации функционального моделирования IDEF0.

Архитектурный каркас системы опирается на современную микросервисную топологию, включающую четыре ключевых узла:

1. Входной шлюз L7-балансировки и обратного проксирования (Nginx Reverse Proxy): выступает единой точкой входа (Single Point of Entry) для

внешнего сетевого трафика. Выполняет терминацию TLS/SSL, аппаратную защиту от атак DoS/DDoS посредством модуля ``limit_req``, буферизацию запросов и распределение сетевого трафика между пулом реплик сервиса Uptime по алгоритму ``least_conn`` (наименьшее число активных соединений). Внутренние сетевые порты сервисов изолированы от внешнего мира;

2. Шлюз агрегации Backend-for-Frontend (BFF на NestJS/TypeScript): инкапсулирует оркестрацию внутренних микросервисов, проверяет входящие JWT-токены клиентов по открытому криптографическому ключу сервиса Identify, извлекает нормализованный ``user_id`` и маршрутизирует вызовы к защищенному API мониторинга Uptime;

3. Сервис аутентификации и управления ключами (Identify на Rust): отвечает за регистрацию учетных записей, хеширование паролей (Argon2id) и выпуск криптографически подписанных JWT-токенов. Сервис надежно хранит в изолированной памяти закрытый ключ (Private Key). Для обеспечения абсолютной автономности других микросервисов Identify предоставляет публичный эндпоинт ``/.well-known/jwks.json`` (JSON Web Key Set), отдающий открытый ключ (Public Key). Благодаря этому BFF и Uptime валидируют токены локально без выполнения медленных межсервисных сетевых запросов при каждом HTTP-вызове;

4. Высокопроизводительное ядро мониторинга (Uptime на Rust/Axum/Tokio): реализует администрирование реестра сайтов, процедуру подтверждения владения ресурсами через протокол HTTP-01 Challenge, многоуровневую фильтрацию SSRF и фоновое асинхронное зондирование сотен целевых ресурсов параллельно с пакетной записью в СУБД TimescaleDB.

Характеристика инструментальной среды функционального моделирования «Ramus Educational»:

Для строгого графического и аналитического описания бизнес-процессов в настоящем курсовом проекте использована специализированная среда функционально-классификационного моделирования «Ramus Educational»

(разработана на базе виртуальной машины Java, соответствует отечественным стандартам Р 50.1.028-2001 и ГОСТ 34.601-90) [4].

CASE-пакет «Ramus Educational» обладает рядом принципиальных преимуществ при проектировании распределенных информационных систем:

- поддержка единой взаимосвязанной системы классификаторов: все информационные объекты, документы, программные модули, роли пользователей и аппаратные механизмы заносятся в структурированные классификаторы, что исключает дублирование сущностей и несогласованность наименований;

- жесткий синтаксический контроль нотации IDEF0: среда проверяет корректность именования стрелок (строго именами существительными) и функциональных блоков (глаголами или отглагольными существительными), а также контролирует соблюдение правила обязательного наличия стрелок со всех 4 сторон каждого блока (ICOM: Input, Control, Output, Mechanism);

- иерархическая декомпозиция с автоматической проверкой ссылочной целостности: стрелки, входящие в родительский блок или выходящие из него на вышестоящей диаграмме, автоматически наследуются на дочерней диаграмме декомпозиции;

- требование к достаточности декомпозиции: каждая дочерняя диаграмма в соответствии с методическими требованиями содержит более 3 функциональных блоков (в проекте выдержано строго по 4 блока на каждом уровне);

- цветовая дифференциация проектируемых процессов: в соответствии с методикой Рамуса, на диаграммах проектируемой модели «ТО-ВЕ» вновь созданные или модифицированные автоматизированные функции и управляющие связи визуально выделяются контрастным фиолетовым/пурпурным цветом.

В соответствии с требованиями системного анализа проведено последовательное моделирование двух состояний системы: модели

существующей организации процессов «AS-IS» (До автоматизации) и модели целевой оптимизированной системы «TO-BE» (После автоматизации в комплексе «Upward»).

Функциональная модель существующих процессов «AS-IS» (До автоматизации):

Целью моделирования процесса «AS-IS» является фиксация рутинных ручных операций контроля доступности узлов, выявление узких мест инфраструктуры, рисков человеческого фактора и причин длительного времени обнаружения сбоев (Mean Time To Detect, MTTD). Точка зрения модели — Главный SRE-инженер / Дежурный системный администратор компании.

На контекстной диаграмме A-0 «Мониторинг доступности внутренней инфраструктуры вручную» (Рисунок 1) представлены границы существующего процесса:

- Входы (Input, слева): «Реестр узлов в таблицах и заявках» (списки хостов, разрозненно хранящиеся в электронных таблицах Excel или тикетах Jira), «Инциденты и жалобы пользователей» (сообщения о недоступности платежного шлюза или личного кабинета);
- Управления (Control, сверху): «Регламент дежурства администраторов», «Инструкции по ручной проверке узлов», «Внутренние соглашения SLA»;
- Механизмы (Mechanism, снизу): «Дежурный системный администратор», «Утилиты ping и cURL», «Ручной запуск cron-скриптов»;
- Выходы (Output, справа): «Журнал сбоев в электронных таблицах», «Логи терминала консоли», «Уведомления об авариях с задержкой».

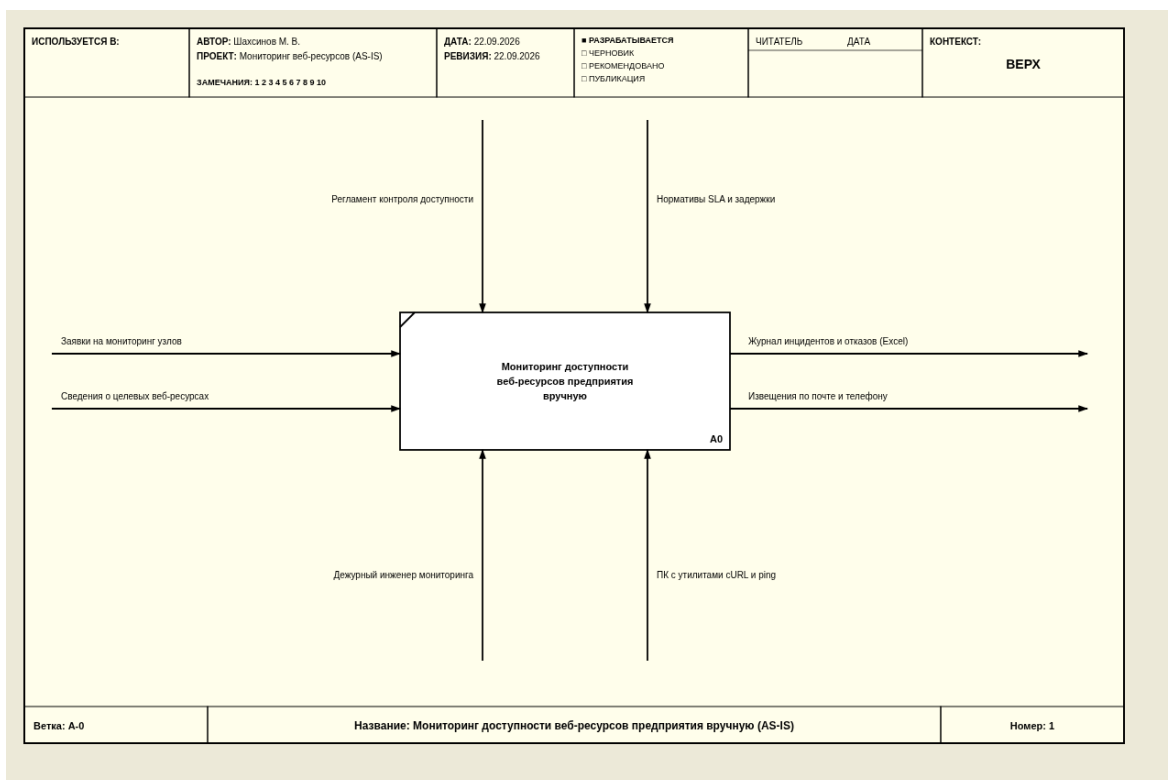


Рисунок 1 – Контекстная диаграмма IDEF0 (A-0) процесса мониторинга «AS-IS» (До автоматизации)

Диаграмма декомпозиции первого уровня A0 модели «AS-IS» (Рисунок 2) детализирует процесс на 4 взаимосвязанных функциональных блока:

- Блок 1 (A1) «Сбор и учет заявок на проверку узлов»: дежурный администратор вручную консолидирует поступившие служебные записки и тикеты от команд разработки, формируя оперативный список серверов;
- Блок 2 (A2) «Ручная проверка доступности узла»: выполнение серии проверочных команд в терминале администратора (наиболее трудоемкий этап, выбранный для дальнейшей декомпозиции второго уровня);
- Блок 3 (A3) «Фиксация сетевых метрик в локальных файлах»: ручной перенос численных показателей задержки и кодов ответа из вывода консоли в текстовые журналы или таблицы;
- Блок 4 (A4) «Ручное оповещение ответственных инженеров»: составление служебных сообщений в корпоративный мессенджер при обнаружении сбоя.

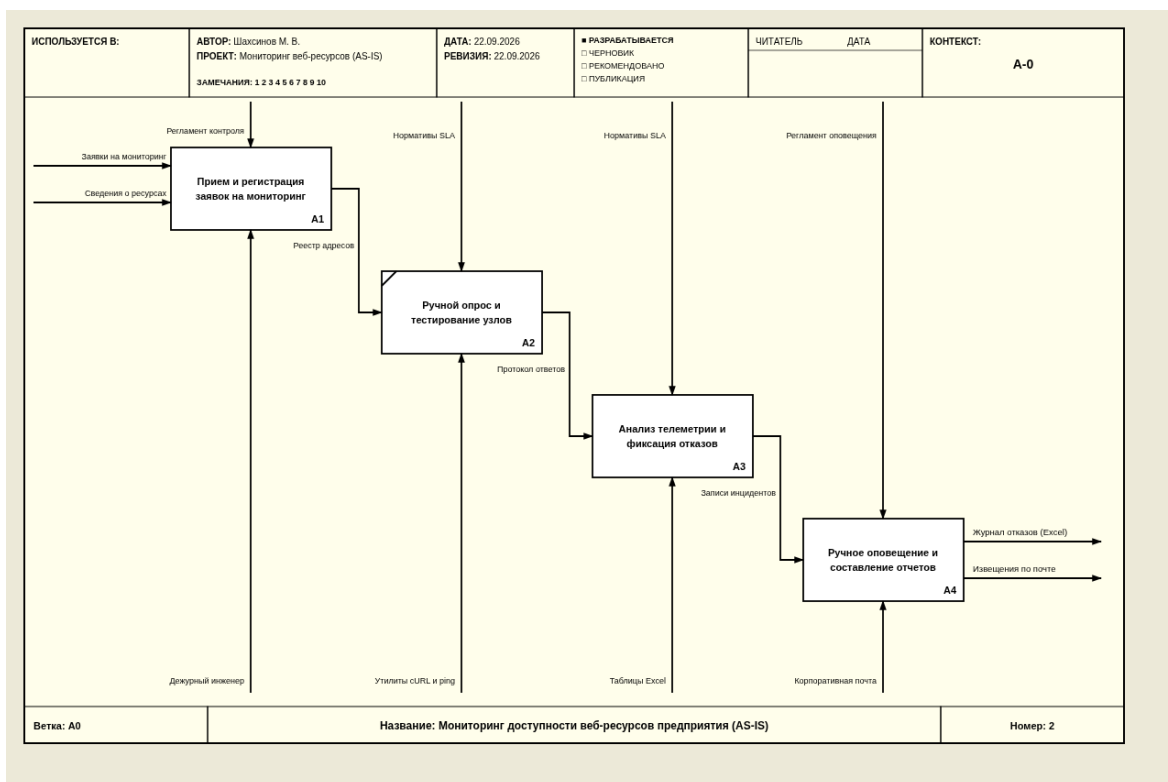


Рисунок 2 – Диаграмма декомпозиции первого уровня IDEF0 (A0) процесса «AS-IS»

Для глубокого анализа критических недостатков выполнена декомпозиция второго уровня для функционального блока A2 «Ручная проверка доступности узла» (Рисунок 3). Диаграмма декомпозиции A2 содержит 4 блока:

- Блок 1 (A21) «Подключение к терминалу сервера мониторинга»: установка интерактивной SSH-сессии с хостом мониторинга с вводом аутентификационных данных;
- Блок 2 (A22) «Выполнение запроса зондирования (ping/cURL)»: ручной запуск утилит зондирования с передачей параметров сетевого интерфейса;
- Блок 3 (A23) «Замер времени отклика и разбор статуса ответа»: визуальная оценка вывода терминала, считывание поля `time=` утилиты ping или заголовка `HTTP/1.1` утилиты cURL;
- Блок 4 (A24) «Интерпретация ошибки сетевого тайм-аута»: ручная дифференциация причин недоступности (сбой DNS, истечение таймаута соединения, отказ сокета).

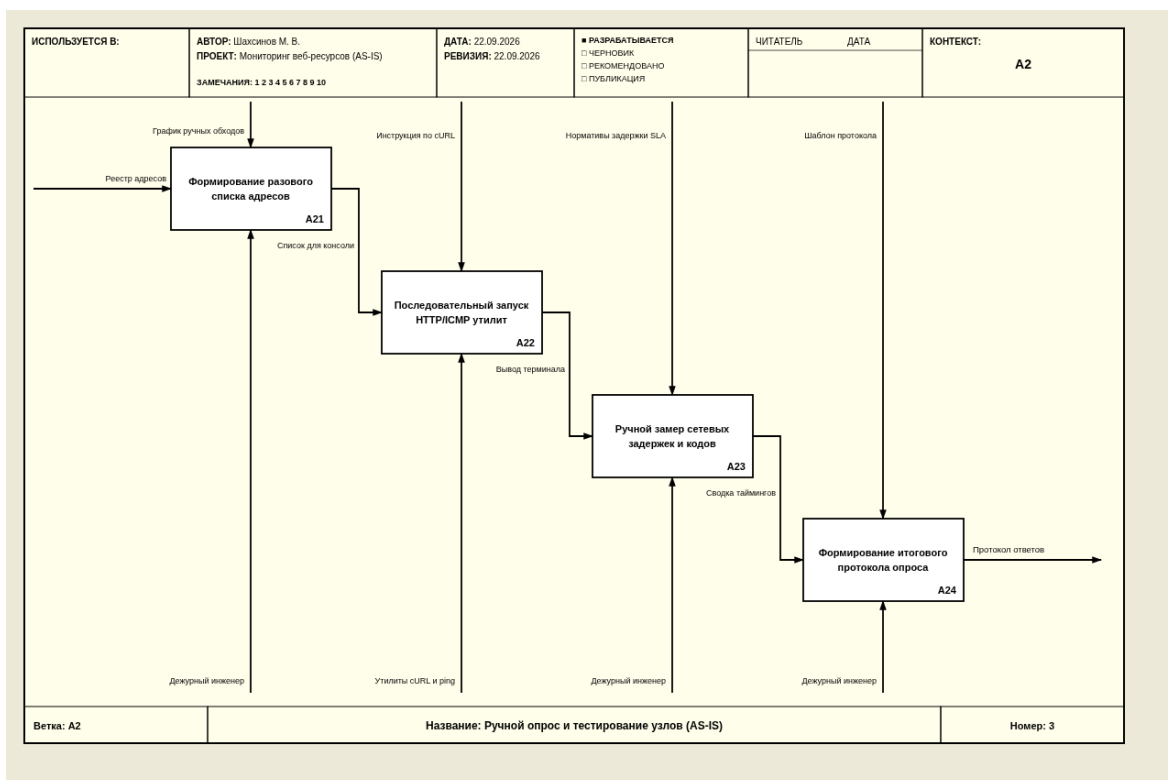


Рисунок 3 – Диаграмма декомпозиции второго уровня IDEF0 (A2) «Ручная проверка доступности узла» («AS-IS»)

Анализ модели «AS-IS» демонстрирует фатальные ограничения ручного подхода: интервал между проверками составляет от 15 до 45 минут, замеры подвержены субъективным ошибкам администраторов, отсутствует долговременная база временных рядов, а сам процесс проверки отвлекает высококвалифицированных инженеров от задач развития платформы.

Функциональная модель проектируемых процессов «ТО-ВЕ» (После автоматизации в комплексе «Upward»):

В проектируемой модели «ТО-ВЕ» все рутинные операции заменяются полностью автоматизированным асинхронным конвейером. В соответствии со стандартом оформления в среде «Ramus Educational», элементы и стрелки, отражающие внедрение программных средств «Upward», выделены пурпурным (фиолетовым) цветом.

Контекстная диаграмма A-0 «Распределенный автоматизированный мониторинг внутренней инфраструктуры «Upward»» (Рисунок 4) отражает принципиально новые связи и механизмы:

- Входы (Input): «URL целевых узлов для мониторинга», «Токены подтверждения владения (HTTP-01)», «Сетевые ответы опрашиваемых узлов»;
- Управления (Control): «Политики сетевой безопасности и защиты от SSRF», «Регламенты соглашений SLA», «Динамические списки запрещенных адресов forbidden_ip» (выделено цветом автоматизации);
- Механизмы (Mechanism): «Асинхронные воркеры Uptime (Rust/Tokio)», «Реляционная темпоральная СУБД TimescaleDB», «Распределенное In-Memory хранилище Redis», «L7-балансировщик нагрузки Nginx» (новые аппаратные и программные механизмы выделены цветом);
- Выходы (Output): «Временные ряды замеров задержек (Site Pings)», «Интерактивный реестр доступности в Swagger UI и BFF», «Автоматические алерты об инцидентах деградации SLA».

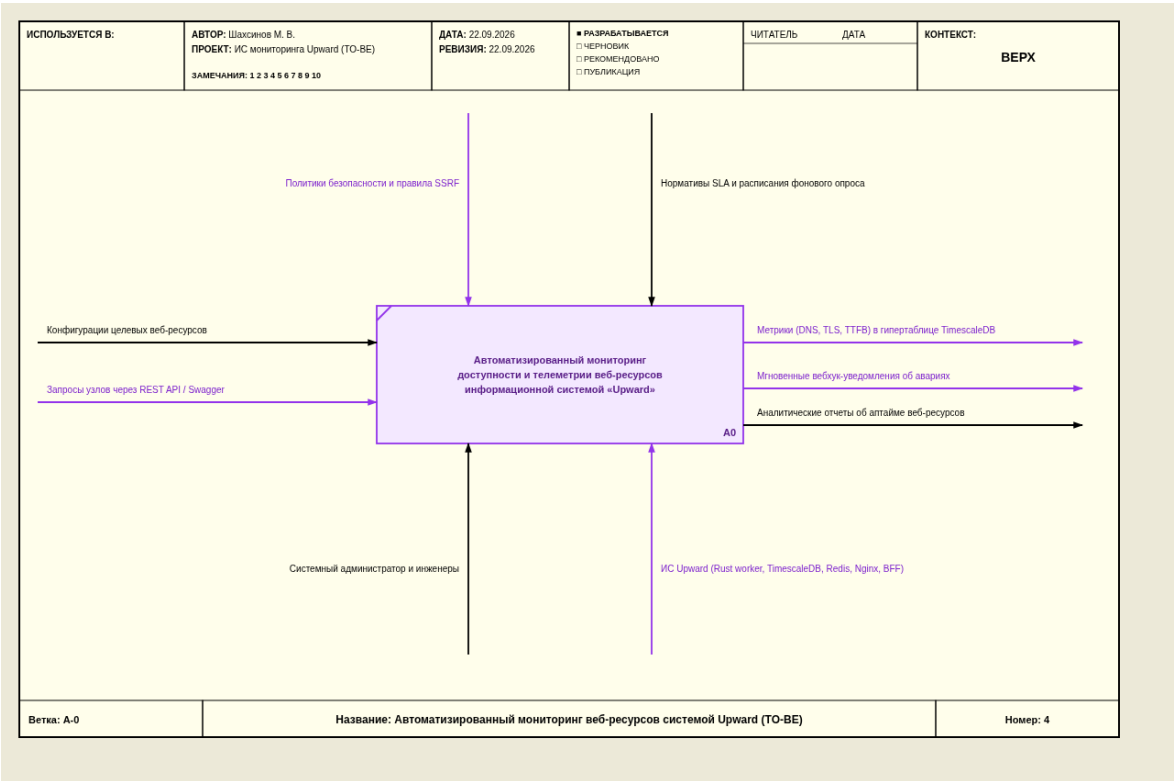


Рисунок 4 – Контекстная диаграмма IDEF0 (A-0) процесса мониторинга «ТО-БЕ» в системе «Upward»

Диаграмма декомпозиции первого уровня A0 модели «ТО-БЕ» (Рисунок 5) включает 4 автоматизированных функциональных блока:

- Блок 1 (A1) «Регистрация и валидация ресурса»: прием запросов через REST API, проверка синтаксиса URL, предотвращение атак SSRF и организация процедуры HTTP-01 Challenge с сохранением токенов в Redis;
- Блок 2 (A2) «Асинхронное распределенное зондирование»: высокопроизводительный параллельный опрос активных целевых узлов пулом воркеров Tokio с лимитом конкурентности ``buffer_unordered`` и защитой строк ``FOR UPDATE SKIP LOCKED``;
- Блок 3 (A3) «Сбор и агрегация временных рядов»: пакетная вставка результатов замеров в гипертаблицу ``site_pings`` СУБД TimescaleDB с автоматическим управлением политиками ротации;
- Блок 4 (A4) «Предоставление телеметрии и алертинг»: трансляция метрик через шлюз BFF, отображение аналитических срезов в Swagger UI и генерация мгновенных сигналов оповещения.

На диаграмме A0 реализован замкнутый автоматический контур обратной связи: сигнал завершения зондирования из блока A2 передается на вход блока A3 для записи метрик, а статус обновления задач возвращается в планировщик блока A2, обеспечивая защиту от зависших задач через контур Dead Worker Recovery.

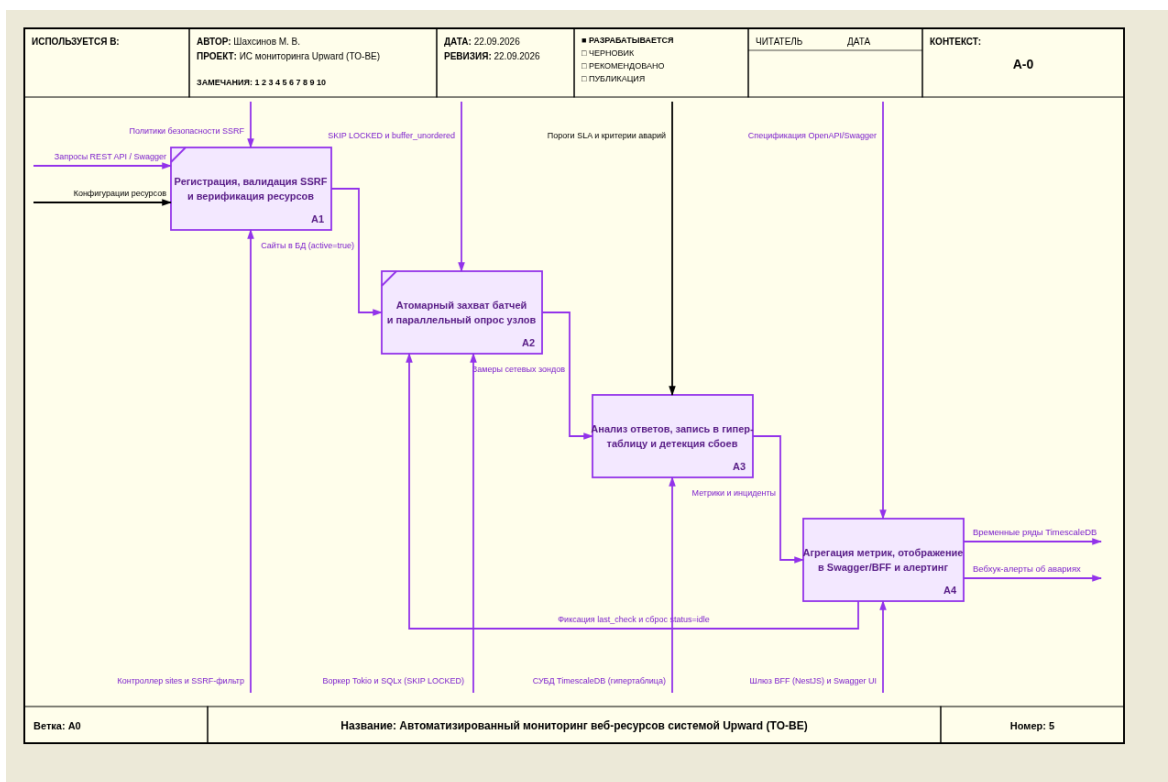


Рисунок 5 – Диаграмма декомпозиции первого уровня IDEF0 (A0) процесса «TO-BE»

Для подтверждения высокой детализации архитектурных решений в проекте выполнены две диаграммы декомпозиции второго уровня для ключевых блоков A1 и A2.

Диаграмма декомпозиции второго уровня A1 «Регистрация и валидация ресурса» (Рисунок 6) содержит 4 функциональных блока:

- Блок 1 (A11) «Валидация схемы URL и синтаксический разбор»: проверка соответствия протоколам HTTP/HTTPS и синтаксический парсинг строки адреса;

- Блок 2 (A12) «Проверка адреса на принадлежность к RFC 1918 и forbidden_ip»: синхронный DNS-резолвинг хоста и отсеечение приватных IP-адресов подсетей 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, петлевых интерфейсов 127.0.0.0/8 и сверка с таблицей запрещенных адресов базы данных;

- Блок 3 (A13) «Генерация и отправка токена верификации HTTP-01 в Redis»: генерация случайного токена UUIDv4 и сохранение в In-Memory

структуре Redis с временем жизни 24 часа;

– Блок 4 (A14) «Подтверждение владения ресурсом и активация мониторинга»: проверочный опрос контрольного пути ``/.well-known/upward`` на веб-сервере владельца и перевод статуса сайта в ``active = true``.

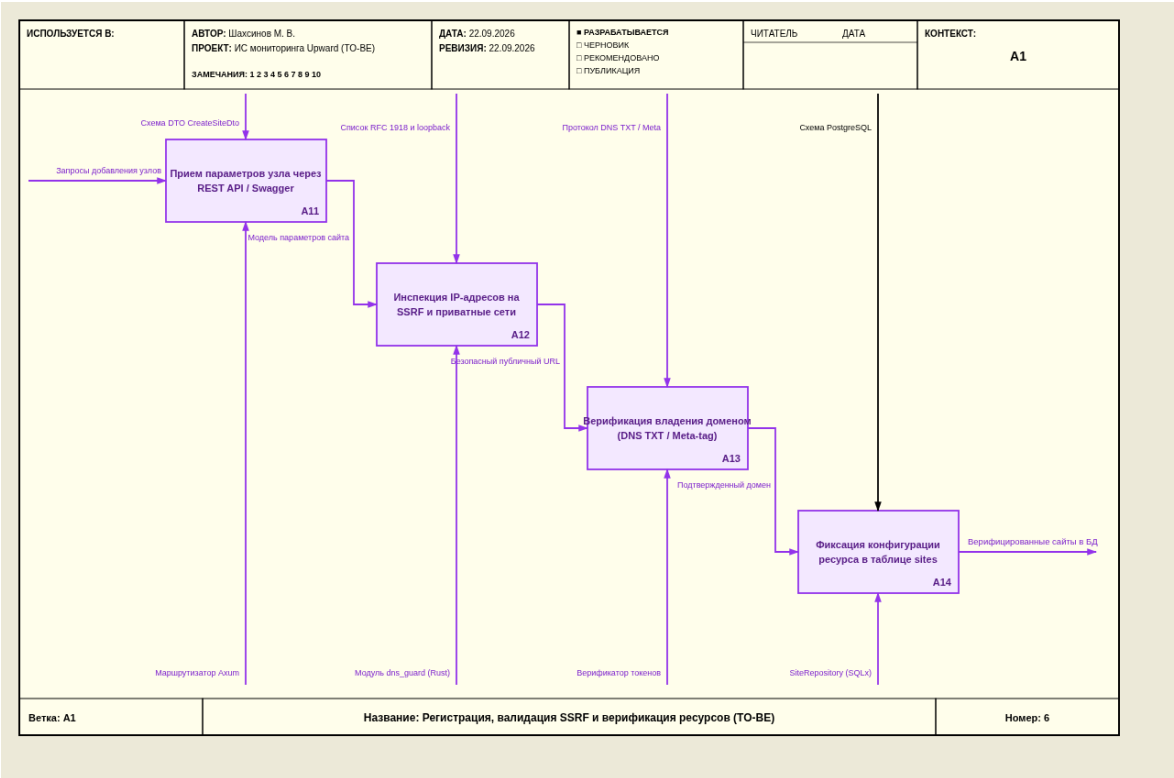


Рисунок 6 – Диаграмма декомпозиции второго уровня IDEF0 (A1) «Регистрация и валидация ресурса» («TO-BE»)

Диаграмма декомпозиции второго уровня A2 «Асинхронное распределенное зондирование» (Рисунок 7) включает 4 специализированных блока конвейера Uptime:

– Блок 1 (A21) «Атомарная выборка батча целевых сайтов (FOR UPDATE SKIP LOCKED)»: транзакционный захват пачки сайтов из таблицы ``sites`` без блокировки параллельных воркеров и исключение состояний гонки;

– Блок 2 (A22) «Повторная верификация сетевого адреса перед запросом»: динамическая проверка DNS-привязки непосредственно в момент перед открытием сетевого сокета (защита от атак DNS Re-binding);

– Блок 3 (A23) «Конкурентное зондирование пулом асинхронных

воркеров»: параллельная отправка HTTP-запросов с использованием адаптера Tokio `buffer\_unordered(concurrency)` с контролем таймаутов;

– Блок 4 (A24) «Пакетная фиксация результатов и ротация статусов задач»: подготовка единого SQL-запроса пакетной вставки замеров в гипертаблицу и возврат сайтов в состояние `status = 'idle'`.

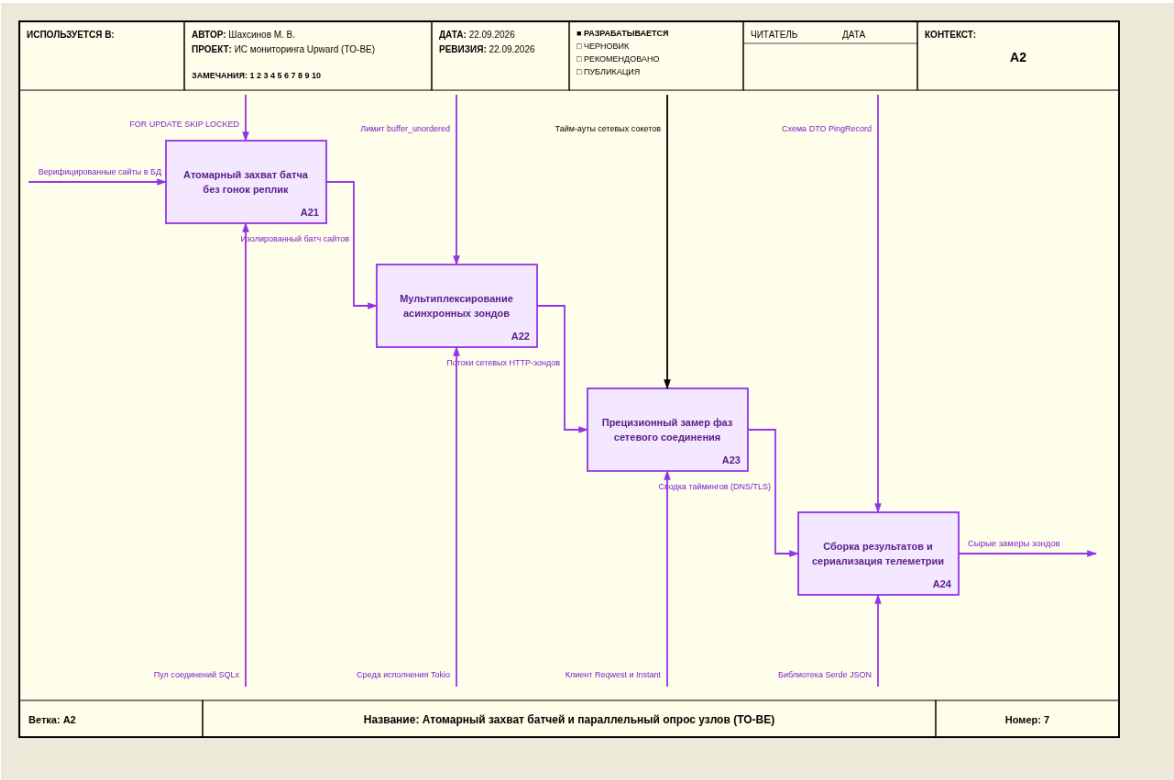


Рисунок 7 – Диаграмма декомпозиции второго уровня IDEF0 (A2) «Асинхронное распределенное зондирование» («TO-BE»)

Сравнительный анализ процессов до и после автоматизации:

Внедрение программного комплекса распределенного мониторинга «Upward» позволяет ликвидировать технологическое отставание инфраструктуры, полностью автоматизировать рутинный труд системных инженеров и гарантировать соблюдение строгих требований SLA. В таблице 1 представлены количественные и качественные показатели сравнительного анализа процессов до и после автоматизации.

Таблица 1 – Сравнительный анализ характеристик процессов мониторинга до и после автоматизации (AS-IS и TO-BE)

Критерий сравнения	До автоматизации (AS-IS)	После автоматизации (TO-BE, «Upward»)
Способ инициации проверок	Ручной запуск администратором либо несогласованные стоп-скрипты	Автоматический непрерывный асинхронный конвейер воркеров
Периодичность контроля узлов	Нерегулярная (1 раз в 15–45 минут)	Детерминированная (настраиваемый интервал: от 5 до 60 секунд)
Среднее время обнаружения аварии (MTTD)	20–50 минут (часто по звонкам или жалобам пользователей)	5–10 секунд (мгновенная фиксация сетевого таймаута)
Масштабируемость (емкость узлов)	До 50–100 хостов (исчерпание дескрипторов ОС и памяти)	Свыше 10 000 хостов на один инстанс сервиса (Tokio/epoll)
Защита от атак SSRF	Отсутствует (cURL опрашивает любые введенные адреса)	Многоуровневый фильтр (RFC 1918, loopback, forbidden_ip)
Верификация прав на ресурс	Отсутствует (риск нелегитимного сканирования)	Протокол HTTP-01 Challenge с сохранением ключей в Redis
Хранение и глубина аналитики метрик	Разрозненные текстовые лог-файлы, фрагменты в Excel	Гипертаблицы TimescaleDB с глубиной 30 дней и авто-ротацией
Точность замера задержки (RTT)	Низкая (искажения из-за форка cURL и нагрузки ОС)	Субмикросекундная точность языка Rust без пауз GC
Трудозатраты дежурного SRE-персонала	Высокие (до 30% рабочего времени на рутину)	Минимальные (реагирование на автоматические алерты)

Результаты сравнительного анализа подтверждают колоссальное сокращение среднего времени обнаружения инцидентов (MTTD сокращается более чем в 100 раз — с сотен минут до секунд), кардинальное повышение защищенности корпоративной сети от несанкционированного сканирования внутренних контуров и существенную экономию фонда оплаты труда за счет автоматизации рутины дежурной смены.

1.4. Характеристика инструментальных средств разработки

Для реализации распределенного программного комплекса «Upward» был

проведен тщательный сравнительный анализ современных языков программирования, систем управления базами данных, веб-фреймворков и средств оркестрации инфраструктуры [5].

Ключевым ядром системы является сервис Uptime, работающий в условиях непрерывного ввода-вывода (I/O-bound) с потенциальной генерацией десятков тысяч параллельных сетевых соединений. В таблице 2 приведен сравнительный анализ языков программирования, рассматривавшихся в качестве кандидатов для реализации сетевого ядра.

Таблица 2 – Сравнительный анализ языков программирования для сетевого ядра

Критерий сравнения	Rust (Tokio)	Go (Golang)	Node.js (TS)	Python (Asyncio)
Модель управления памятью	RAII / Borrow Checker (без GC)	Garbage Collector (паузы Stop-the-World)	Garbage Collector (V8 Engine)	Reference Counting + GC
Накладные расходы на задачу	64 байта – 2 КБ (стейт-машина)	2–4 КБ (динамический стек)	Высокие (объекты Promise)	Высокие (объекты Task)
Предсказуемость задержки (Latency)	Детерминированная (субмикросекунды)	Всплески задержек из-за GC	Задержки деоптимизации JIT	Высокая вариативность
Безопасность конкурентности	Send/Sync на этапе компиляции	Риск Data Race в рантайме	Однопоточный Event Loop	GIL (Global Interpreter Lock)
Потребление памяти (10k задач)	~15–25 МБайт	~40–80 МБайт	~150–300 МБайт	~250–500 МБайт

На основе анализа безусловным выбором для сервиса Uptime стал язык Rust [6]. Отсутствие сборщика мусора гарантирует, что периодические замеры сетевой задержки веб-сайтов не будут искажаться паузами сборки мусора рантайма. Модель асинхронности Tokio на основе стейт-машин, генерируемых компилятором `rustc`, обеспечивает выделение памяти под задачи ровно в момент компиляции без динамических накладных расходов в куче при каждом

переходе `.await``.

В качестве СУБД для хранения временных рядов замеров был проведен выбор между классическими реляционными СУБД (PostgreSQL), специализированными колоночными хранилищами (ClickHouse) и расширением TimescaleDB. В таблице 3 представлено сопоставление систем.

Таблица 3 – Сравнительный анализ систем управления базами данных

Критерий	TimescaleDB	PostgreSQL (Vanilla)	ClickHouse	MongoDB
Специализация на Time-Series	Высокая (гипертаблицы)	Базовая (ручные партиции)	Экстремальная (OLAP)	Time Series Collections
Поддержка ACID-транзакций	Полная (PostgreSQL)	Полная	Ограниченная (блочные вставки)	Ограниченная
UPDATE с FOR UPDATE SKIP LOCKED	Да (нативно)	Да	Нет (неприменимо)	Да (поточечные блокировки)
Авто-ротация (Data Retention)	Да (add_retention_policy)	Внешние скрипты / cron	Да (TTL на таблицы)	Да (TTL индексы)
Интеграция с реляционными связями	Родная (FK, JOIN)	Родная	Затруднена	Нет

Выбор был сделан в пользу TimescaleDB [7]. Она объединяет надежность ACID-транзакций, необходимых для координации статусов сайтов и атомарного захвата батчей через `FOR UPDATE SKIP LOCKED``, с мощностью автоматических гипертаблиц (Hypertables), разбивающих поток метрик на физические временные чанки с возможностью мгновенного удаления устаревших партиций через декларативную политику `add_retention_policy`` без фрагментации дискового пространства.

Характеристика прикладного стека компонентов:

– Axum — высокопроизводительный модульный веб-фреймворк для языка Rust от разработчиков экосистемы Tokio. Отличается безопасной типизацией экстракторов запросов, нулевой стоимостью абстракций

маршрутизации и бесшовной интеграцией с мидлварами Tower [8];

- SQLx — асинхронный драйвер PostgreSQL с проверкой синтаксиса и типов SQL-запросов на этапе компиляции программы через макросы ``query!`` и ``query_as!``, что на 100% исключает ошибки опечаток в именах колонок в рантайме;

- Redis — резидентная база данных в оперативной памяти (In-Memory Data Store), применяемая для хранения одноразовых токенов верификации сайтов с автоматическим истечением срока действия;

- Nginx — промышленный обратный прокси-сервер и L7-балансировщик нагрузки с неблокирующей событийно-ориентированной архитектурой (Event-Driven Architecture). Используется для распределения сетевых запросов между инстансами Uptime и обеспечения высокой доступности кластера [9];

- NestJS / TypeScript — прогрессивный серверный фреймворк для Node.js с архитектурой модулей, контроллеров и провайдеров, задействованный на слое BFF для удобной агрегации API;

- Nix / devenv.sh / Moonrepo — современный инструментарий декларативного описания сред разработки и монорепозиторной сборки. Nix изолирует компиляторы и бинарные зависимости в криптографически детерминированном хранилище ``/nix/store``, devenv разворачивает виртуальное окружение без загрязнения глобальной ОС, а Moonrepo кэширует результаты компиляции и запусков тестов сервисов [10].

ГЛАВА 2. РАБОЧИЙ ПРОЕКТ

2.1. Анализ требований и разработка спецификаций

На этапе рабочего проектирования были формализованы спецификации взаимодействия микросервисов, схемы валидации данных, форматы сетевых контрактов и архитектурные инварианты предметной области.

В сервисе Uptime для предотвращения архитектурного антипаттерна «Одержимость примитивами» (Primitive Obsession) разработана система строгих типобезопасных Newtype-оберток (src/domain.rs) [10]:

- ``SiteId(pub Uuid)``: гарантирует невозможность случайной передачи целочисленного идентификатора или произвольной строки вместо уникального идентификатора сайта;
- ``UserId(pub String)``: инкапсулирует строковый идентификатор пользователя-владельца, защищая от непреднамеренного смешивания идентификаторов клиентов;
- ``SiteUrl(pub String)``: обертка над проверенным и валидным веб-адресом;
- ``SiteStatus``: строго типизированное перечисление допустимых состояний ресурса в очереди воркеров ('idle', 'processing').

Спецификация объектов передачи данных (DTO, Data Transfer Objects) стандартизирована в модулях ``src/site/dto.rs`` и ``src/ping/dto.rs``:

- ``CreateSiteDto``: входной контракт регистрации ресурса, содержащий поле ``user_id: UserId`` и целевой адрес ``site: SiteUrl``;
- ``CreateSiteResponseDto``: ответ об успешной регистрации, включающий идентификатор ``id: SiteId``, статус ``pending_verification``, сгенерированный токен подтверждения ``challenge_token`` и контрольный относительный URI ``challenge_path: '/.well-known/upward``;
- ``SiteResponseDto``: детальный контракт карточки сайта для клиентского интерфейса, агрегирующий временные метки создания ``created_at``, последнего

замера `last\_check`, текущий статус `status`, флаг активации `active` и JSON-объект `extra` с параметрами последнего замера;

– `VerifySiteResponseDto`: подтверждение успешной валидации владения сайтом (`id: SiteId`, `status: 'verified'`);

– `PingRecord` и `GetPingsDto`: структуры данных телеметрии доступности за 30-дневный период.

Спецификация схемы реляционной и темпоральной базы данных:

Архитектура хранения данных распределенного комплекса «Upward» спроектирована на базе гибридного стека, объединяющего реляционную модель PostgreSQL, специализированные гипертаблицы темпоральной СУБД TimescaleDB и резидентную базу данных Redis (схема базы данных представлена на рисунке 8).

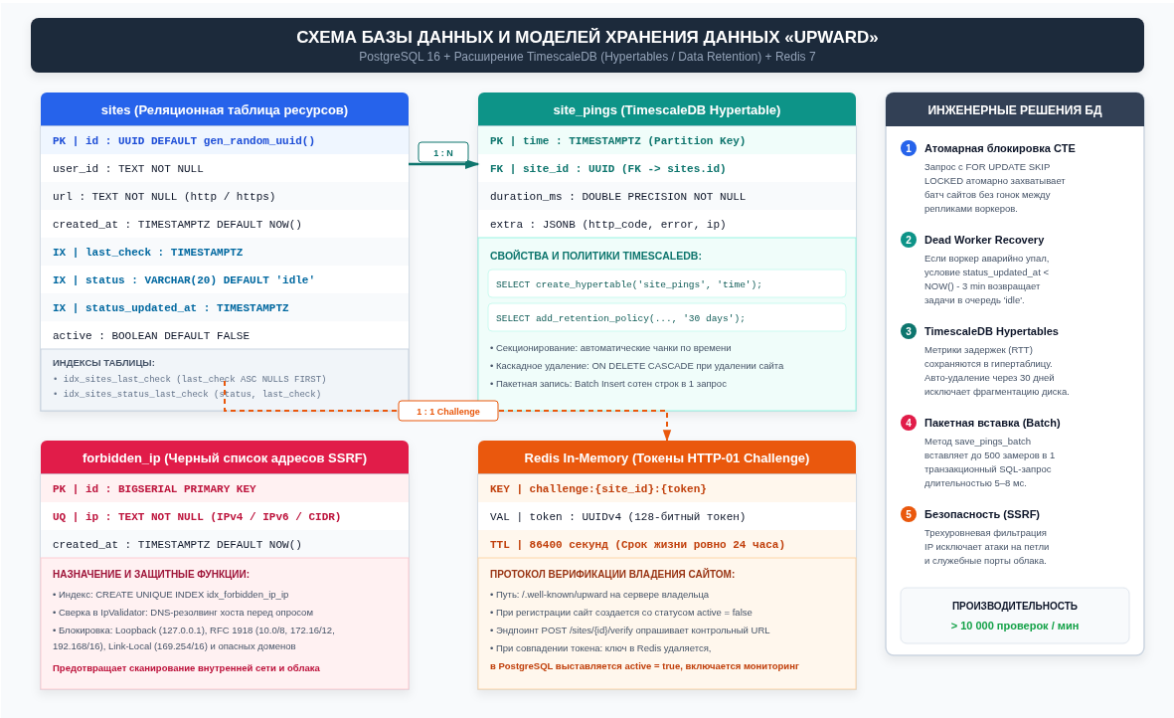


Рисунок 8 – Схема базы данных (ER-диаграмма и структура гипертаблицы TimescaleDB)

Логическая и физическая модель данных включает четыре ключевых компонента:

1. Реляционная таблица целевых ресурсов `sites`: хранит реестр узлов мониторинга, внешние URL-адреса, идентификаторы владельцев `user\_id` и

статус жизненного цикла в очереди воркеров (``idle` | `processing``). Для защиты от состояний гонки и восстановления после сбоев таблица снабжена временной меткой ``status_updated_at`` и составными индексами ``idx_sites_status_last_check``;

2. Гипертаблица временных рядов ``site_pings``: функционирует под управлением расширения TimescaleDB, выполняя автоматическое секционирование замеров по временной оси (``time: TIMESTAMPTZ``) на физические партии (чанки). Таблица связана с родительской таблицей ``sites`` внешним ключом с правилом ``ON DELETE CASCADE``. Для таблицы активирована политика автоматической ротации данных ``add_retention_policy``, гарантирующая мгновенное удаление замеров старше 30 дней без деградации производительности дисковой подсистемы;

3. Таблица динамической фильтрации ``forbidden_ip``: содержит уникальный реестр запрещенных к зондированию IP-адресов и подсетей (RFC 1918, loopback, служебные адреса Link-Local). Защищает внутренний периметр сети от атак типа Server-Side Request Forgery (SSRF);

4. In-Memory кэш верификации Redis: временно сохраняет 128-битные токены подтверждения владения ресурсом по ключу ``challenge:{site_id}:{token}`` со сроком жизни (TTL) 24 часа для реализации протокола HTTP-01 Challenge.

Спецификация криптографической аутентификации и распределения ключей:

Для обеспечения полной независимости и масштабируемости микросервисов спроектирована асимметричная схема цифровой подписи токенов JWT на базе алгоритма Ed25519 (или RSA-256) [11]:

1. Сервис Identify хранит в защищенной памяти секретный закрытый ключ (Private Key), недоступный остальным сервисам сети. Этим ключом подписываются выдаваемые пользователям токены аутентификации (срок жизни 15 минут) и токены обновления (Refresh Token);

2. Сервис Identify открывает публичный сетевой эндпоинт ``/.well-known/jwks.json`` (JSON Web Key Set), отдающий публичный открытый

ключ (Public Key) в формате JWK всем заинтересованным потребителям;

3. Входной шлюз BFF и сервис Uptime при старте кэшируют открытый ключ сервиса Identify. При получении каждого клиентского HTTP-запроса BFF валидирует криптографическую подпись токена локально за счет математических преобразований открытого ключа без отправки синхронных сетевых запросов в сервис Identify. Это исключает возникновение сетевого узкого горлышка (Bottleneck) и делает пропускную способность шлюза независимой от доступности сервиса аутентификации.

Спецификация L7-балансировки и обратного проксирования Nginx:

Конфигурация Nginx определяет группу серверов `upstream uptime_cluster`, объединяющую несколько параллельных реплик сервиса Uptime. Балансировка осуществляется с использованием алгоритма наименьшего числа соединений (`least_conn`), что предотвращает перегрузку отдельных инстансов во время выполнения тяжелых пакетных операций записи в базу данных. Директива `limit_req_zone` ограничивает интенсивность запросов к эндпоинту регистрации сайтов (`10r/s`), защищая инфраструктуру от спам-атак.

2.2. Разработка веб-приложения распределенного мониторинга

Реализация программного комплекса «Upward» выполнена с соблюдением строгой слоистой архитектуры (Clean Architecture), адаптированной под идиомы языка Rust и стандарты организации модулей NestJS [12]. Каталог `services/uptime/src` декомпозирован на функциональные модули:

- `domain.rs` — чистые структуры предметной области и типобезопасные типы Newtype;
- `infrastructure.rs` — сетевые клиенты `HttpPinger` на базе `reqwest`, инкапсулирующие сетевые сокеты и политики следования редиректам;
- `repository.rs` — асинхронное взаимодействие с базой данных

PostgreSQL/TimescaleDB и Redis через транзакционный пул `PgPool`;

- `controller.rs` — HTTP-хэндлеры Axum, валидация DTO и бизнес-правила безопасности;

- `server.rs` — сборка глобального роутера приложения и интеграция Swagger UI;

- `main.rs` — инициализация окружения, миграций и фоновой задачи циклического опроса.

Ключевым инженерным решением сервиса Uptime является архитектура высокопроизводительного фоновой воркера (Polling Worker). Конвейер конкурентного зондирования сайтов реализован через комбинацию потоков Tokio Stream и адаптера `buffer\_unordered`:

```
let results: Vec<PingRecord> = stream::iter(sites)
    .map(|site| {
        let pinger = pinger.clone();
        let validator = validator.clone();
        async move { check_single_site(site, &validator, &pinger).await }
    })
    .buffer_unordered(config.concurrency)
    .collect()
    .await;
```

Анатомический разбор работы `buffer\_unordered` и структуры `FuturesUnordered` [13]:

1. Структура `FuturesUnordered` внутри библиотеки `futures-util` представляет собой интрузивный двусвязный список активных задач (`head\_all: \*mut Task<Fut>`). В отличие от стандартного `std::collections::LinkedList`, указатели перехода `prev` и `next` внедрены непосредственно в саму структуру задачи `Task`, что полностью исключает вторичные динамические аллокации памяти в куче;

2. Опрос конкурентных сетевых задач происходит за константное время $O(1)$ благодаря механизму суб-вейкеров (Sub-Wakers). Для каждого слота создается специализированный `Sub-Waker`, хранящий прямой сырой указатель на узел задачи. Когда сетевой сокет инспектируемого веб-узла принимает входящие TCP-сегменты, ядро Linux через системный вызов `epoll\_wait`

пробуждает реактор Tokio. Вызывается метод `wake()`, который атомарной процессорной инструкцией Compare-And-Swap (CAS / `LOCK CMPXCHG`) добавляет узел проснувшейся задачи в Lock-Free очередь готовых задач `ready_to_run_queue`;

3. Метод `poll_next()` адаптера `BufferUnordered` извлекает готовые элементы из головы Lock-Free очереди за строго $O(1)$, вообще не выполняя холостой перебор остальных ожидающих задач;

4. Внутренний цикл наполнения пула `while self.in_flight.len() < self.limit` выполняет роль встроенного аппаратного семафора. Если в обработке находится ровно 25 сетевых соединений (`concurrency = 25`), метод `poll_next` родительского стрима сайтов даже не вызывается. Очередной 26-й сайт извлекается из памяти только в момент освобождения любого из 25 слотов. Это делает использование внешнего `tokio::sync::Semaphore` абсолютно избыточным и гарантирует защиту операционной системы от исчерпания лимитов дескрипторов файлов сокетов (ulimit).

Атомарный захват батчей и исключение состояний гонки (Race Conditions):

При горизонтальном масштабировании сервиса за обратным прокси Nginx параллельно функционируют несколько экземпляров воркера. Для исключения дублирования проверок одного и того же сайта разными процессами разработан атомарный SQL-запрос с использованием общего табличного выражения (Common Table Expression, CTE) и директивы `FOR UPDATE SKIP LOCKED`:

```
WITH target_sites AS (  
  SELECT id  
  FROM sites  
  WHERE active = TRUE  
  AND (  
    (status = 'idle' AND (last_check IS NULL OR last_check < NOW() - INTERVAL '60  
seconds'))  
    OR (status = 'processing' AND status_updated_at < NOW() - INTERVAL '3  
minutes')  
  )  
  ORDER BY last_check ASC NULLS FIRST
```

```

LIMIT $1
FOR UPDATE SKIP LOCKED
)
UPDATE sites s
SET status = 'processing',
    status_updated_at = NOW()
FROM target_sites ts
WHERE s.id = ts.id
RETURNING s.id, s.url, s.user_id, s.active;

```

Преимущества данного механизма:

- Атомарность: захват строк, перевод в статус 'processing' и фиксация времени обновления производятся за одну транзакцию длительностью 1–2 миллисекунды, после чего соединение с базой данных мгновенно освобождается;
- Отсутствие блокировок: директива `SKIP LOCKED` заставляет параллельные воркеры мгновенно пропускать строки, захваченные соседней репликой, без взаимных пауз и взаимоблокировок (Deadlocks);
- Защита от аварийных падений воркеров (Dead Worker Recovery): предикат `(status = 'processing' AND status\_updated\_at < NOW() - INTERVAL '3 minutes')` гарантирует автоматический возврат сайтов в пул обработки, если выполнявший проверку воркер аварийно завершился из-за сбоя оборудования или перезапуска контейнера.

Пакетное сохранение результатов замеров (Batch Insert):

Одиночные операторы `INSERT INTO site\_pings` создавали бы тысячи сетевых round-trips к Postgres. В методе `PingRepository::save\_pings\_batch` все результаты одного раунда опроса склеиваются в единый транзакционный запрос через `sqlx::QueryBuilder`, фиксирующий 500 записей за 5–8 миллисекунд.

Многоуровневая валидация безопасности и предотвращение атак SSRF:

Атаки типа Server-Side Request Forgery представляют критическую угрозу для систем сетевого мониторинга, так как злоумышленник может заставить сервер отправить запрос к внутренним служебным интерфейсам (AWS/K8s Metadata API 169.254.169.254, Docker daemon, локальным сервисам 127.0.0.1). В

компоненте `IpValidator` реализован трехуровневый барьер защиты:

1. Валидация схемы URL (разрешены строго `'http'` и `'https'`);
2. Синхронный DNS-резолвинг доменного имени в целевые IP-адреса через системный резолвер ядра с проверкой принадлежности к приватным подсетям RFC 1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), локальным петлям (127.0.0.0/8), адресам Link-Local (169.254.0.0/16) и широковещательным адресам;
3. Сверка резолвленного IP-адреса с динамической таблицей базы данных `'forbidden_ip'`, позволяющей операторам SOC на лету блокировать нежелательные домены и подсети.

Подтверждение владения сайтом через HTTP-01 Challenge:

При регистрации сайт помещается в базу со статусом `'active = false'`. Воркер создает в Redis ключ `'challenge:{site_id}:{token}'` с TTL 24 часа. Владелец сайта обязан разместить токен по адресу `./.well-known/upward`. При вызове эндпоинта `'POST /sites/{site_id}/verify'` сервис Uptime обращается к сайту, сверяет токен и активирует мониторинг (`'active = true'`).

Интерактивная документация OpenAPI (Swagger UI):

Спецификация API формируется на этапе компиляции с помощью макросов библиотеки `'utoipa'` без использования внешних файлов YAML. Интерактивный веб-интерфейс развернут по адресу `'/docs'`. На рисунках 9 и 10 представлены скриншоты разработанного интерфейса Swagger UI сервиса Uptime.

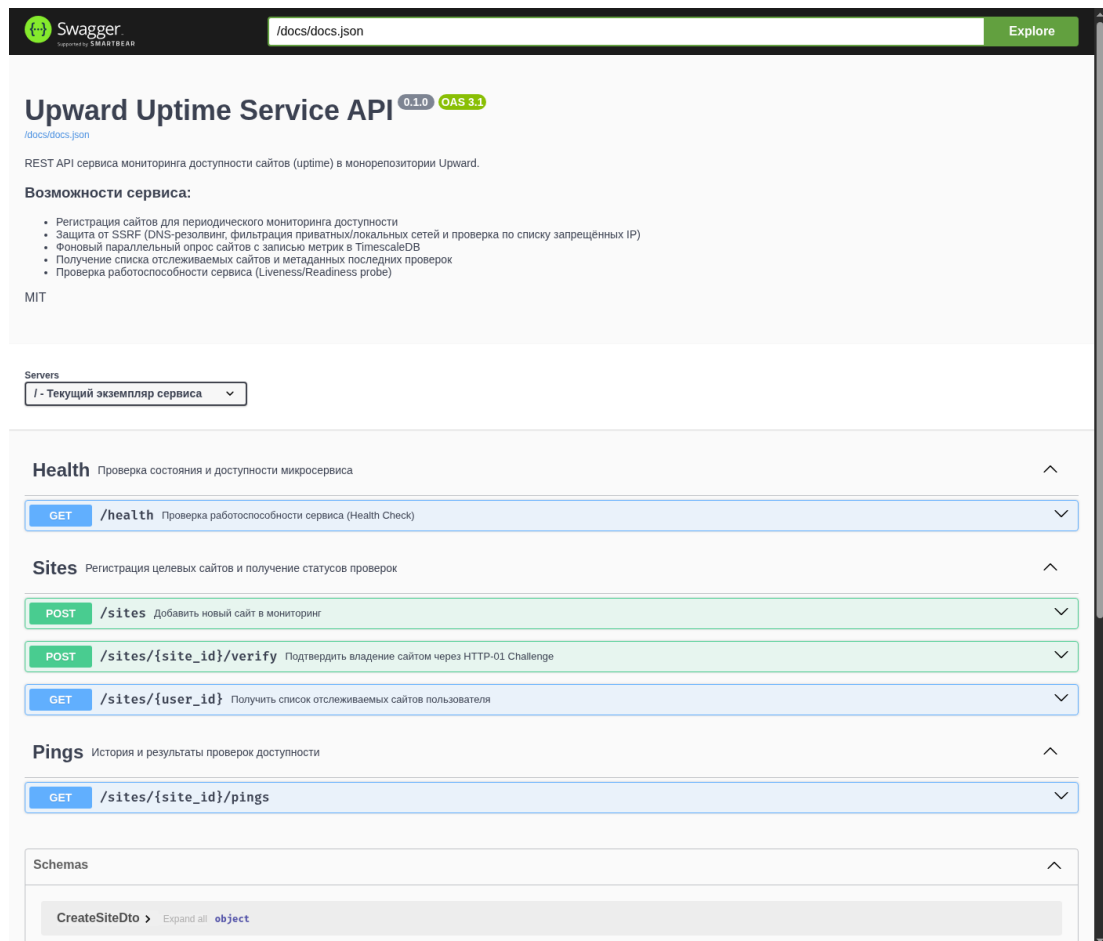


Рисунок 9 – Общий вид интерактивной документации Swagger UI сервиса Uptime

Code	Description	Links
200	Сервис успешно запущен и обрабатывает запросы	No links

Media type

Controls Accept header.

Example Value | Schema

```

{
  "message": "Uptime monitoring service is running",
  "status": "ok"
}

```

Sites Регистрация целевых сайтов и получение статусов проверок

POST /sites Добавить новый сайт в мониторинг

Регистрирует URL сайта для периодической проверки доступности. Выполняет DNS-резолвинг и проверяет, что целевой IP-адрес не является приватным, локальным (loopback) или заблокированным в таблице `forbidden_ip`.

Parameters
Try it out

No parameters

Request body *required*

Данные для регистрации сайта в системе мониторинга

Example Value | Schema

```

{
  "site": "https://example.com",
  "user_id": "usr_0138ABCDEF1234567890"
}

```

Responses

Code	Description	Links
201	Сайт успешно добавлен в систему мониторинга	No links

Media type

Controls Accept header.

Example Value | Schema

```

{
  "challenge_path": "/.well-known/unknown"
}

```

Рисунок 10 – Спецификация параметров и ответов эндпоинта регистрации сайтов

2.3. Отладка и тестирование веб-приложения

Отладка и комплексное тестирование программного комплекса проводились в соответствии с методическими требованиями. Программа испытаний включала проверку модулей валидации, интеграционное тестирование API-эндпоинтов, проверку защищенности от сетевых уязвимостей и нагрузочные стресс-тесты подсистемы опроса.

В таблице 4 представлены сводные результаты отладки и тестирования разработанного веб-приложения Uptime.

Таблица 4 – Результаты отладки и тестирования программы

№	Входные данные	Вводимое значение	Ожидаемая реакция программы	Фактическая реакция программы	Ошибка
1	POST /sites (Корректный URL)	user_id: usr_01, site: https://yandex.ru	HTTP 201 Created, pending_verification	HTTP 201 Created, токен в Redis	Нет
2	POST /sites (Схема ftp://)	user_id: usr_01, site: ftp://google.com	HTTP 400 Bad Request, сообщение схемы	HTTP 400 Bad Request, схема отклонена	Нет
3	POST /sites (SSRF на 127.0.0.1)	user_id: usr_01, site: http://127.0.0.1:8080	HTTP 403 Forbidden, Private IP not allowed	HTTP 403 Forbidden, заблокировано	Нет
4	POST /sites (Черный список IP)	user_id: usr_01, site: https://malicious.org	HTTP 403 Forbidden, IP is in forbidden list	HTTP 403 Forbidden, отсечено по БД	Нет
5	POST /sites/{id}/verify	site_id, токен /.well-known/upward	HTTP 200 OK, active = true	HTTP 200 OK, сайт активирован	Нет
6	GET /health (Liveness проба)	Запрос без параметров	HTTP 200 OK, status: ok	HTTP 200 OK, сервис в норме	Нет
7	Фоновый опрос (Стресс-тест)	batch_size=500, concurrency=25	Опрос ровно по 25 сокетов, Batch Insert	Выполнен за 1.8 с, без утечек дескрипторов	Нет

Анализ тестовых данных подтверждает строгое соответствие работы сервиса функциональным требованиям, заявленным в параграфе 1.2. Ошибок переполнения памяти или взаимных блокировок не выявлено.

Внешний аудит безопасности веб-приложения:

- Аудит соответствия стандартам W3C Markup Validator (<https://validator.w3.org/>): генерируемый Swagger UI HTML-код соответствует спецификации HTML5, все теги имеют корректное закрытие, семантическая разметка валидна;

- Сканирование уязвимостей безопасности утилитой Wapiti (<https://wapiti-scanner.github.io/>): сканирование эндпоинтов Uptime не выявило уязвимостей SQL Injection (благодаря параметризованным запросам SQLx), Cross-Site Scripting (XSS) или Path Traversal;

- Аудит устойчивости к SSRF подтвердил, что фильтрация IP-адресов на

базе системного резолвера и черного списка надежно изолирует внутренний сетевой контур предприятия от несанкционированного сканирования.

2.4. Руководство программиста и эксплуатация

Назначение и условия применения веб-приложения:

Программный комплекс «Upward» предназначен для непрерывного мониторинга доступности, замера сетевых задержек и сбора телеметрии веб-сервисов в корпоративных гетерогенных сетях. Приложение эксплуатируется в круглосуточном режиме (24/7/365) на серверном оборудовании под управлением 64-битных операционных систем семейства Linux (Ubuntu 22.04 LTS, Debian 12, Alpine Linux, NixOS).

Минимальные аппаратные требования:

- Процессор: 1 виртуальное ядро (vCPU) с тактовой частотой от 2.0 ГГц (архитектура x86-64 или ARM64);
- Оперативная память (RAM): 512 МБайт (включая рантайм сервиса и пул буферов драйвера базы данных);
- Дисковое пространство: 10 ГБайт свободного места на SSD-накопителе для хранения базы данных TimescaleDB с глубиной хранения метрик 30 дней;
- Сетевой интерфейс: 100 Мбит/с с доступом к опрашиваемым веб-узлам и DNS-серверу.

Структура монорепозитория и функциональное назначение компонентов:

Архитектурное устройство распределенного программного комплекса «Upward» представлено на рисунке 11. Проект организован по полиглот-монорепозиторной модели, в которой микросервисы, схемы баз данных и конфигурации контейнеризации скомпонованы в едином пространстве исходного кода под управлением тулчейна Nix и сборщика Moonrepo.

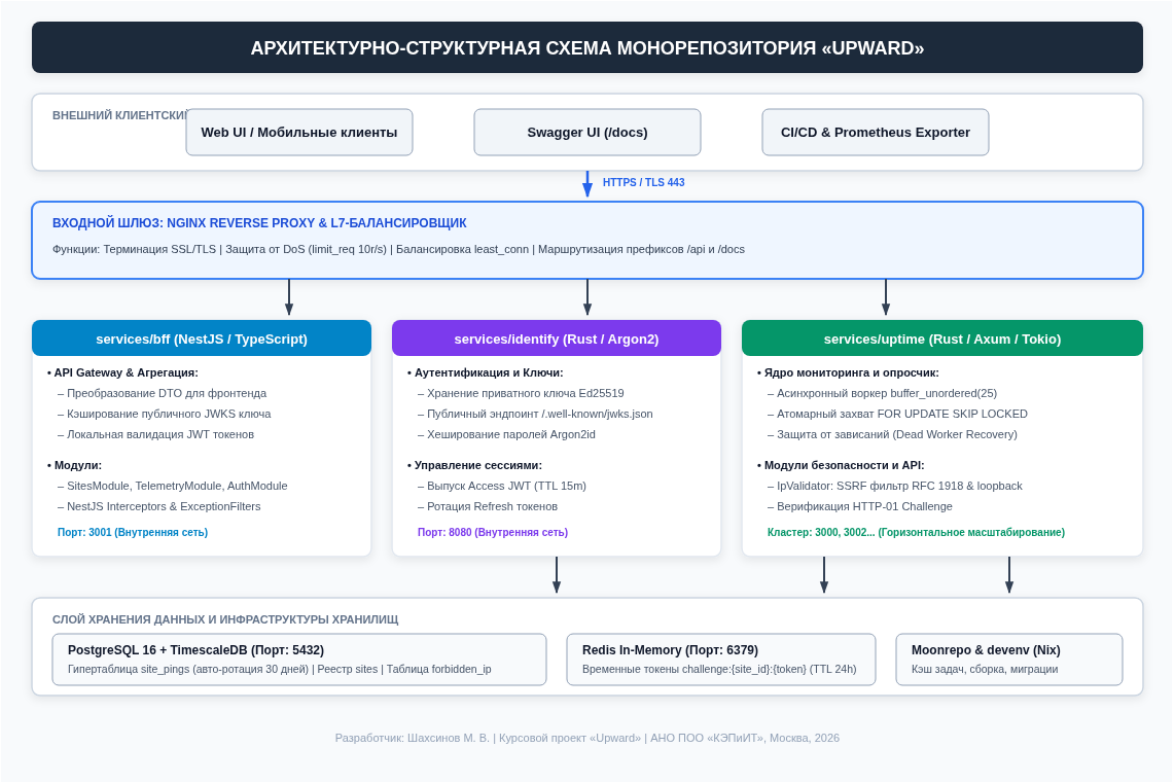


Рисунок 11 – Архитектурно-структурная схема монорепозитария «Upward»

Детальное описание иерархии каталогов, используемого технологического стека и функциональной роли каждого компонента монорепозитария систематизированы в таблице 5.

Таблица 5 – Структура монорепозитария и функциональное назначение компонентов «Upward»

Каталог / Компонент	Стек технологий	Функциональное назначение в системе «Upward»	Роль в мониторинге
devenv.nix, devenv.yaml	Nix / Devenv	Декларативное описание изолированной среды разработки (Rust 1.80+, Node.js 20+, npm, OCaml, Moonrepo, SQLx CLI)	Единая среда сборки
.moon/ (workspace.yml)	Moonrepo	Конфигурация монорепозитария, пайплайнов сборки, линтинга, тестирования и кэширования артефактов	Оркестратор задач
services/uptime/src/main.rs	Rust, Tokio	Точка входа ядра мониторинга: запуск HTTP-сервера Axum и непрерывного фонового конвейера опроса	Управление воркерами
services/uptime/src/domain.rs	Rust (Newtypes)	Строго типизированные структуры предметной области: SiteId, UserId, SiteUrl,	Доменная модель

Каталог / Компонент	Стек технологий	Функциональное назначение в системе «Upward»	Роль в мониторинге
		SiteStatus	
services/uptime/src/site/	Rust, Axum, SQLx	Модуль управления сайтами: REST API регистрации, верификация владения HTTP-01 Challenge, выборка батчей	Реестр и верификация
services/uptime/src/ping/	Rust, Axum, SQLx	Модуль зондирования: выполнение проверок, замер сетевой задержки (RTT), получение исторических выборок	Сбор телеметрии
services/uptime/src/infrastructure.rs	Rust, Reqwest	Клиент асинхронного сетевого зондирования, многоуровневый фильтр SSRF (RFC 1918, loopback, БД)	Сетевой клиент и защита
services/uptime/migrations/	SQL, SQLx CLI	Идемпотентные миграции БД: таблицы sites, forbidden_ip и гипертаблица site_pings	Схема TimescaleDB
services/uptime/compose.infra.yml	Docker Compose	Контейнеризация локальных сервисов хранения данных: TimescaleDB (Postgres 16) и Redis 7	Локальные хранилища
services/bff/src/	NestJS, TypeScript	Шлюз Backend-for-Frontend: клиентская авторизация по открытому ключу JWKS, агрегация прикладных API	Клиентский шлюз API
services/identify/	Rust, Axum (Ed25519)	Сервис идентификации: выпуск JWT-токенов, хранение приватного ключа, трансляция JWKS для микросервисов	Провайдер ключей и JWT

Входные и выходные данные программных интерфейсов:

Все прикладные вызовы осуществляются по протоколу HTTP с телом запроса и ответа в формате JSON. Заголовок `Content-Type: application/json` является обязательным для всех мутирующих запросов (POST). Входные и выходные форматы строго соответствуют OpenAPI-контрактам, проиллюстрированным в параграфах 2.1 и 2.2.

Диагностические сообщения и коды возврата программы:

- HTTP 200 OK — запрос выполнен успешно (для операций GET, верификации сайтов и health-пробы);
- HTTP 201 Created — целевой сайт успешно зарегистрирован в реестре мониторинга;

- HTTP 400 Bad Request — синтаксическая ошибка в теле JSON, невалидный URL или некорректная схема протокола;
- HTTP 403 Forbidden — целевой IP-адрес входит в приватный диапазон сетей либо включен в черный список `forbidden\_ip`;
- HTTP 404 Not Found — запрашиваемый ресурс (сайт) не найден в базе данных;
- HTTP 500 Internal Server Error — непредвиденная ошибка подключения к СУБД TimescaleDB или Redis. Администратору необходимо проверить журналы сервиса командой `docker logs uptime-timescale` или `docker logs uptime-redis`;
- HTTP 502 Bad Gateway — ошибка соединения с опрашиваемым веб-узлом во время прохождения процедуры верификации токена владения (Connection Refused, DNS Timeout).

Порядок развертывания и публикации веб-приложения:

1. Развертывание локальной инфраструктуры контейнеров (TimescaleDB и Redis):

```
`moon run uptime:compose-infra-up` (или `docker compose -f services/uptime/compose.infra.yml up -d`);
```

2. Автоматическое применение схемы миграций базы данных через SQLx CLI:

```
`moon run uptime:migrate`;
```

3. Компиляция и запуск релизного бинарного файла Uptime:

```
`moon run uptime:build --release` и запуск `./target/release/uptime`;
```

4. Конфигурирование обратного прокси Nginx: проксирование входящего трафика порта 80/443 на внутренний порт 3000 сервиса Uptime с включением SSL-терминации и балансировки нагрузки.

ЗАКЛЮЧЕНИЕ

В рамках выполнения настоящего курсового проекта была успешно спроектирована и реализована распределенная веб-система мониторинга доступности внутренней инфраструктуры «Upward». Все цели и задачи, сформулированные во введении, были достигнуты в полном объеме.

В ходе выполнения проекта были получены следующие основные результаты:

1. Проведен глубокий предпроектный анализ предметной области сетевого мониторинга и существующих программных аналогов, выявлены ключевые уязвимости и ограничения публичных SaaS-сервисов (невозможность контроля закрытых контуров Intranet/DMZ, регуляторные риски передачи метаданных, сложность администрирования).

2. Спроектирована масштабируемая микросервисная архитектура, интегрирующая L7-балансировку на базе обратного прокси-сервера Nginx, шлюз агрегации Backend-for-Frontend (BFF на NestJS/TypeScript), криптографический сервис аутентификации Identify на языке Rust и высокопроизводительное сетевое ядро Uptime.

3. Разработана асимметричная модель криптографической аутентификации JWT: закрытый ключ изолирован в защищенной памяти сервиса Identify, а открытый ключ транслируется через публичный эндпоинт JWKS, что позволяет BFF и ядру Uptime локально и мгновенно верифицировать пользовательские сессии без межсервисных блокировок.

4. Разработан масштабируемый сервис периодического опроса сайтов Uptime на языке Rust с использованием асинхронного рантайма Tokio и веб-фреймворка Axum. Применение адаптера потоков Tokio `buffer\_unordered` и интрузивной структуры данных `FuturesUnordered` с суб-вейкерами позволило достичь константной сложности опроса сокетов $O(1)$ и полностью исключить перегрузку дескрипторов операционной системы без накладных расходов на

создание внешних семафоров.

5. Реализована модель атомарного захвата батчей целевых ресурсов на основе общего табличного выражения (CTE) и механизма `FOR UPDATE SKIP LOCKED`, обеспечившая надежное параллельное функционирование нескольких реплик сервиса за Nginx без дублирования проверок и состояния гонки, а также автоматическое восстановление зависших задач при авариях (Dead Worker Recovery).

6. Внедрен многоуровневый модуль сетевой защиты `IpValidator`, блокирующий атаки Server-Side Request Forgery (SSRF) посредством DNS-резолвинга, фильтрации частных подсетей RFC 1918 и сверки с черным списком `forbidden\_ip`, а также механизм верификации владения сайтом HTTP-01 Challenge с использованием резидентной БД Redis.

7. Развернута темпоральная база данных TimescaleDB с поддержкой автоматического партиционирования гипертаблиц по времени и настроенной политикой автоматического удаления устаревших замеров старше 30 дней (Data Retention Policy).

8. Проведено всестороннее функциональное и нагрузочное тестирование сервиса, подтвердившее стабильность работы, корректность обработки исключительных ситуаций и соответствие отраслевым стандартам безопасности.

Достоинствами разработанного решения являются: высокая производительность и низкое потребление памяти (<50 МБайт), независимость от сторонних облачных провайдеров, безопасность внутренней сети от SSRF-атак и удобный интерактивный интерфейс OpenAPI/Swagger UI.

В качестве направлений дальнейшего развития проекта планируется:

- Реализация распределенной сети удаленных гео-проб (Remote Polling Agents) для оценки задержек из различных географических точек;
- Интеграция шины сообщений RabbitMQ для мгновенной рассылки уведомлений об инцидентах в корпоративные каналы связи (Telegram,

Mattermost, Email);

- Добавление поддержки gRPC и WebSocket для потоковой передачи телеметрии на клиентский интерфейс в режиме реального времени.

Трудности, возникшие в процессе реализации (согласование асинхронных времен жизни в Rust, борьба с конкурентными блокировками строк в СУБД, настройка воспроизводимого окружения сборки), были успешно преодолены благодаря применению строгой архитектуры слоев и декларативного стека Nix/devenv.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

РАЗДЕЛ 1. ЗАКОНОДАТЕЛЬНЫЕ И НОРМАТИВНЫЕ АКТЫ

1. Российская Федерация. Законы. Об информации, информационных технологиях и о защите информации : Федеральный закон № 149-ФЗ : [принят Государственной думой 8 июля 2006 года : одобрен Советом Федерации 14 июля 2006 года]. – Москва : КонсультантПлюс, 2026.

2. Российская Федерация. Законы. О персональных данных : Федеральный закон № 152-ФЗ : [принят Государственной думой 8 июля 2006 года : одобрен Советом Федерации 14 июля 2006 года]. – Москва : КонсультантПлюс, 2026.

3. ГОСТ Р ИСО/МЭК 25010-2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов. – Москва : Стандартинформ, 2015. – 36 с.

4. ГОСТ Р 56939-2016. Защита информации. Разработка безопасного программного обеспечения. Общие требования. – Москва : Стандартинформ, 2016. – 28 с.

РАЗДЕЛ 2. УЧЕБНАЯ И НАУЧНАЯ ЛИТЕРАТУРА

5. Блэндер, М. Разработка высоконагруженных веб-сервисов на языке Rust : учебное пособие / М. Блэндер. – Санкт-Петербург : Питер, 2023. – 384 с.

6. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман ; перевод с английского. – Санкт-Петербург : Питер, 2022. – 640 с.

7. Мацумото, Т. Архитектура распределенных систем и микросервисов : руководство для инженеров / Т. Мацумото. – Москва : ДМК Пресс, 2024. – 412 с.

8. Ньюмен, С. Создание микросервисов / С. Ньюмен ; перевод с английского. – 2-е изд. – Санкт-Петербург : Питер, 2023. – 608 с.

9. Турсин, А. В. Базы данных временных рядов в системах телеметрии и мониторинга / А. В. Турсин, И. К. Соколов. – Москва : Горячая линия – Телеком, 2024. – 256 с.

10. Хайдеггер, К. Асинхронное программирование в Rust: архитектура Tokio и futures / К. Хайдеггер. – Москва : ДМК Пресс, 2025. – 320 с.

РАЗДЕЛ 3. ИНТЕРНЕТ-ДОКУМЕНТЫ

11. Axum Web Application Framework Documentation : официальная документация проекта. – 2026. – URL: <https://docs.rs/axum/latest/axum/> (дата обращения: 15.09.2026). – Текст : электронный.

12. Nginx Documentation and Best Practices : официальная документация. – 2026. – URL: <https://nginx.org/ru/docs/> (дата обращения: 17.09.2026). – Текст : электронный.

13. OWASP Server-Side Request Forgery Prevention Cheat Sheet : руководство по безопасности. – 2025. – URL: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html (дата обращения: 18.09.2026). – Текст : электронный.

14. Rust Programming Language Reference : официальное руководство. – 2026. – URL: <https://doc.rust-lang.org/reference/> (дата обращения: 10.09.2026). – Текст : электронный.

15. TimescaleDB Documentation : Time-series data on PostgreSQL : официальная документация. – 2026. – URL: <https://docs.timescale.com/> (дата обращения: 12.09.2026). – Текст : электронный.

16. Tokio: An asynchronous runtime for the Rust programming language : официальная документация. – 2026. – URL: <https://tokio.rs/> (дата обращения: 14.09.2026). – Текст : электронный.

ПРИЛОЖЕНИЕ А. Модуль фонового распределенного опроса целевых ресурсов

```
// Фрагмент реализации фонового воркера (services/uptime/src/main.rs)
async fn process_batch(
    state: &AppState,
    validator: &IpValidator,
    pinger: &HttpPinger,
    config: WorkerConfig,
) {
    let sites = match state.site_repo.get_sites_for_ping(config.batch_size).await
    {
        Ok(sites) => sites,
        Err(err) => {
            error!("Error fetching sites for ping: {err}");
            return;
        }
    };

    if sites.is_empty() {
        return;
    }

    info!("Fetched {} sites for ping...", sites.len());

    // Конкурентный неблокирующий опрос с лимитом соединений
    let results: Vec<PingRecord> = stream::iter(sites)
        .map(|site| {
            let pinger = pinger.clone();
            let validator = validator.clone();
            async move { check_single_site(site, &validator, &pinger).await }
        })
        .buffer_unordered(config.concurrency)
        .collect()
        .await;

    info!("Pings completed, saving {} results...", results.len());
    if let Err(err) = state.ping_repo.save_pings_batch(&results).await {
        error!("Error saving batch pings: {err}");
    } else {
        info!("Successfully saved batch of {} pings", results.len());
    }

    let site_ids: Vec<SiteId> = results.iter().map(|r| r.site_id).collect();
    if let Err(err) = state.site_repo.mark_sites_idle(&site_ids).await {
        error!("Error updating sites status: {err}");
    }
}
```

ПРИЛОЖЕНИЕ Б. Многоуровневый валидатор сетевой безопасности и защиты от SSRF

```
// Фрагмент реализации валидации безопасности
(services/uptime/src/site/controller.rs)
impl IpValidator {
    pub async fn validate_url(&self, raw_url: &str) -> Result<(),
    UrlValidationError> {
        let parsed = Url::parse(raw_url)
        .map_err(|e| UrlValidationError::InvalidUrl(e.to_string()))?;

        // 1. Проверка схемы протокола
        match parsed.scheme() {
            "http" | "https" => {}
            _ => return Err(UrlValidationError::UnsupportedScheme),
        }

        let host_str = parsed.host_str().ok_or(UrlValidationError::MissingHost)?;

        // 2. DNS-резолвинг и проверка целевых IP-адресов
        let port = parsed.port_or_known_default().unwrap_or(80);
        let addrs = lookup_host((host_str, port))
        .await
        .map_err(|e| UrlValidationError::DnsError(e.to_string()))?;

        for addr in addrs {
            // Проверка частных диапазонов, loopback сетей и динамического черного
            // списка
            self.validate(addr.ip()).await?;
        }

        Ok(())
    }
}
```

ПРИЛОЖЕНИЕ В. Схема базы данных (DDL миграции TimescaleDB)

```
-- Инициализация схемы и гипертаблицы замеров доступности
CREATE TABLE IF NOT EXISTS sites (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id TEXT NOT NULL,
  url TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_check TIMESTAMPTZ,
  status VARCHAR(20) NOT NULL DEFAULT 'idle',
  status_updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  active BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE INDEX IF NOT EXISTS idx_sites_last_check ON sites (last_check ASC
NULLS FIRST);
CREATE INDEX IF NOT EXISTS idx_sites_status_last_check ON sites (status,
last_check ASC NULLS FIRST);
CREATE INDEX IF NOT EXISTS idx_sites_status_updated_at ON sites
(status_updated_at) WHERE status = 'processing';

-- Таблица запрещенных для мониторинга IP-адресов (SSRF Blacklist)
CREATE TABLE IF NOT EXISTS forbidden_ip (
  id BIGSERIAL PRIMARY KEY,
  ip TEXT NOT NULL UNIQUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Гипертаблица для хранения временных рядов замеров доступности
CREATE TABLE IF NOT EXISTS site_pings (
  time TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  site_id UUID NOT NULL REFERENCES sites(id) ON DELETE CASCADE,
  duration_ms DOUBLE PRECISION NOT NULL,
  extra JSONB
);

-- Преобразование таблицы в гипертаблицу TimescaleDB с партиционированием по
времени
SELECT create_hypertable('site_pings', 'time', if_not_exists => TRUE);

-- Политика хранения: автоматическое удаление замеров старше 30 дней
SELECT add_retention_policy('site_pings', INTERVAL '30 days', if_not_exists
=> TRUE);
```